Lab 4: S-Expressions

CAS CS 320: Concepts of Programming Languages

October 21, 2026

S-expressions are simple syntactic representations of heirarchical data—the individual data are called ***atoms***. S-expressions are commonly used to serialize data, along the lines of JSON or XML (e.g., Dune configurations are S-expressions) and as the concrete syntax of PLs (e.g., Lisp programs are S-expressions). They're represented in code by the following ADT:

```
type sexpr =
  | Atom of String
  | List of sexpr list
```

For example, the S-expression `(abc (d e f) ((g ((h i)) jk) lm))` is represented by the following value:

```
List
  [
    Atom "abc";
    List [Atom "d"; Atom "e"; Atom "f"];
    List
      [
        List
          [
            Atom "g";
            List [List [Atom "h"; Atom "i"]];
            Atom "jk";
          ];
        Atom "lm"
      ]
  ]
```

Parsing S-expressions. In this lab you'll be implementing a parser for S-expressions. You'll be using this parser (and variations of it) throughout this course, so it's important that you understand how it works.

Exercise 1. *The lexer for S-expressions is given to you. Read through the implementation, make sure you understand what it's doing.*

Exercise 2. *Implement the function*

```
val of_tokens : Sexpr.Token.t list -> Sexpr.t option
```

so that `Sexpr.of_tokens ts` *is the S-expression represented by the token list* `ts`.

There are a number of ways to approach building an S-expressions parser. One particularly elegant way[1] is to define two helper functions, one of type:

```
Sexpr.Token.t list -> (Sexpr.t, Sexpr.Token.t list) option
```

that attempts to parse the *longest* prefix of its input into an S-expression (and fails if no such prefix exists) and one of type:

```
Sexpr.Token.t list -> (Sexpr.t list, Sexpr.Token.t list)
```

that does the same thing but for *lists* of S-expressions; this one will never fail but may produce an empty list. Here are some examples of the expected behavior of these functions.

```
let parse_one (_ts : token list) : (sexpr * token list) option =
  assert false

let _ = assert
  (parse_one (tokens_of_string "(abc (d e)) fgh")
   = Some
     ( List [Atom "abc"; List [Atom "d"; Atom "e"]]
     , [W "fgh"]))

let _ = assert
  (parse_one (tokens_of_string "(abc (d e) fgh")
   = None

let parse_many (_ts : token list) : (sexpr list * token list) =
  assert false

let _ = assert
  (parse_many (tokens_of_string "(abc (d e)) fgh")
```

[1] In my opinion.

```
   = ( [
         List [Atom "abc";
         List [Atom "d"; Atom "e"]];
         [Atom "fgh"];
       ]
     , []))

let _ = assert
  (parse_many (tokens_of_string "(abc ")
   = ([], [L; W "abc"]))
```

In particular, it may be useful to define these functions *in terms each other* (i.e., mutually).

Using S-expressions as a concrete syntax for a PL. Once we have an S-expressions parser, we can quickly spin up parsers for PLs with S-expressions for concrete syntax. We just need to map S-expressions to values of an ADT representing the abstract syntax of the language. For example, consider the following BNF grammar for arithmetic S-expressions:

```
<expr> ::= ( <op> <expr> <expr> ) | <int>
 <op>  ::= + | - | *
```

which can be represented abstractly in code by the following ADT:

```
type op = Add | Sub | Mul
type expr =
  | Bop of op * expr * expr
  | Int of int
```

Exercise 3. *Implement the function*

```
val of_sexpr : Sexpr.t -> Expr.t option
```

so that `Expr.of_sexpr se` *is the arithmetic expression represented by the S-expression* `se`*.*

Exercise 4. ***(Challenge)*** *Look up the function* `Option.bind` *in the OCaml standard library documentation. Describe what it does. Then read about* `let*` *binding operators, either in the OCaml manual or elsewhere. Make use of them in your implementation of* `of_sexpr`*.*